

Despliegue de Software

Contenerización y orquestación de una API REST con Docker y Kubernetes

APERTURA

25 de septiembre de 2026

ENTREGA

4 de octubre de 2026

Objetivo general

Desarrollar habilidades prácticas para contenerizar y desplegar una API REST en Docker y Kubernetes, independientemente del lenguaje, framework o entorno de desarrollo seleccionado por el equipo.

Contexto del caso

Cada grupo debe seleccionar o desarrollar una API REST con una funcionalidad mínima significativa (por ejemplo: usuarios, productos, reservas o reportes). El objetivo de la práctica es el despliegue, por lo que la tecnología de implementación queda a elección del equipo. La aplicación será ejecutada primero en Docker y luego en Kubernetes local.

Libertad tecnológica

- Lenguaje y framework: libre elección. Ejemplos: .NET/ASP.NET Core, Java/Spring Boot, Node.js/Express o NestJS, Python/FastAPI o Flask, entre otros.
- Entorno de desarrollo: Visual Studio Community 2026, Visual Studio Code, IntelliJ IDEA, PyCharm u otro IDE/editor que el equipo domine.
- Los videos y demostraciones del docente podrán realizarse con .NET y Visual Studio como referencia, pero no constituyen una restricción tecnológica para la práctica.
- La API debe poder verificarse de forma clara mediante Swagger/OpenAPI, un cliente como Postman o un mecanismo equivalente que permita demostrar sus endpoints.



Parte 1 • Despliegue local con Docker Desktop

El grupo debe contenerizar su API y ejecutarla exitosamente en un contenedor Docker local. La tecnología utilizada para implementar la API no cambia los objetivos de esta parte.

Condición de trabajo

El equipo puede trabajar desde el IDE/editor de su preferencia y utilizar su terminal integrada o una consola del sistema. Se espera que pueda explicar qué hace cada instrucción utilizada en el proceso de construcción y ejecución de la imagen.

Producto esperado

- Dockerfile válido para la tecnología seleccionada.
- Imagen Docker generada correctamente y con una etiqueta de versión clara (por ejemplo: v1).
- Contenedor Docker ejecutándose localmente.
- Puerto publicado y API accesible desde un navegador, Swagger/OpenAPI, Postman o mecanismo equivalente.

Evidencias mínimas

- Captura del contenedor ejecutándose en Docker Desktop.
- Captura de la imagen Docker generada.
- Captura que demuestre el acceso funcional a la API.
- Captura de la terminal o herramienta utilizada mostrando, al menos, una validación del contenedor o de la imagen.

Secuencia sugerida de validación

```
docker build -t practica2-api:v1 .  
docker images  
docker run -d -p <puerto-host>:<puerto-api> --name practica2-api practica2-api:v1  
docker ps
```

Importante

Los puertos y comandos pueden variar según la tecnología utilizada. El equipo debe documentar en el README el comando real que empleó, los puertos publicados y la forma exacta de probar la API.

Parte 2 • Orquestación con Kubernetes local

Después de verificar la ejecución en Docker, la misma imagen debe desplegarse en Kubernetes local mediante manifiestos YAML.

Requisitos mínimos

- Un Deployment funcional para la API.
- Un Service que permita acceder a la aplicación mediante NodePort o port-forward.
- Uso coherente de labels y selectors.
- Acceso funcional a la API a través del servicio Kubernetes, usando el mecanismo de prueba elegido por el equipo.

Evidencias mínimas

- Captura del resultado de `kubectl get pods -n practica2`.
- Captura del resultado de `kubectl get svc -n practica2`.
- Captura que demuestre el acceso funcional a la API desplegada en Kubernetes.
- Captura o fragmento legible de los manifiestos YAML utilizados.

Plus técnico • configuración básica de buenas prácticas

Alcanzable y aplicable a cualquier tecnología

Además del Deployment y Service, el grupo debe aplicar estas tres configuraciones simples: 1) crear y trabajar en un namespace propio llamado `practica2`; 2) definir requests y limits de CPU/memoria para el contenedor; y 3) usar una etiqueta de imagen versionada (`v1`, `v2`, etc.) junto con `imagePullPolicy: IfNotPresent`.

Ejemplo orientativo de recursos

```
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "250m"
    memory: "256Mi"
```

Criterio

No se evaluará una tecnología específica ni que estos valores representen una configuración de producción perfecta. Se evaluará que la solución funcione, que los manifiestos sean válidos y que el grupo pueda explicar las decisiones adoptadas.

Entregables por grupo

1

Repositorio GitHub exclusivo

- Debe contener todo el código fuente, Dockerfile, manifiestos YAML y documentación necesaria para reproducir el despliegue.
- Agregar como colaborador al docente con el usuario: oalarconpe.
- README.md con nombres completos de los integrantes, tecnología utilizada, enlace al video, puertos empleados y comandos mínimos de ejecución.

2

Video de implementación

- Demostrar la API ejecutándose en Docker local.
- Demostrar la API desplegada en Kubernetes local.
- Todos los integrantes deben participar en la explicación técnica.
- El video debe estar publicado en YouTube y enlazado desde el README.

3

Evidencias visuales

- Incluir las capturas solicitadas para Docker y Kubernetes.
- Las imágenes deben permitir identificar claramente los comandos, recursos y resultados obtenidos.

4

Reflexión técnica · máximo una página

- ¿Cómo abordaron el proceso de despliegue?
- ¿Qué errores encontraron y cómo los resolvieron?
- ¿Cómo se distribuyeron las responsabilidades del equipo?
- ¿Qué decisiones relacionadas con la tecnología seleccionada influyeron en el Dockerfile o en la configuración del despliegue?

Checklist antes de entregar

- | | |
|---|---|
| ☐ La API funciona dentro de Docker. | ☐ Namespace practica2 configurado. |
| ☐ La imagen tiene una etiqueta de versión. | ☐ Requests y limits definidos. |
| ☐ Kubernetes tiene pods en estado Running. | ☐ Repositorio compartido con oalarconpe. |
| ☐ El Service permite acceder a la API. | ☐ README, video, evidencias y reflexión incluidos. |

Resultado esperado

El grupo debe demostrar dominio del ciclo completo de despliegue: construir una imagen reproducible, ejecutar la API en Docker, orquestarla con Kubernetes y documentar el proceso de forma que otro equipo pueda comprenderlo y repetirlo, independientemente de la tecnología seleccionada.

Videos de apoyo en YouTube

Docker y Kubernetes

Orientación

Los videos muestran el procedimiento utilizando .NET como tecnología de referencia. Los estudiantes pueden adaptar el mismo proceso de contenerización y orquestación a la tecnología y al entorno de desarrollo seleccionados para su proyecto.

1

Contenedores parte 1

Introducción práctica a la contenerización y preparación inicial del proyecto.

[Ver video en YouTube](#)

2

Contenedores parte 2

Continuación del trabajo con imágenes y contenedores para ejecutar la aplicación.

[Ver video en YouTube](#)

3

Kubernetes parte 1

Primer acercamiento al despliegue y configuración de la aplicación en Kubernetes.

[Ver video en YouTube](#)

4

Kubernetes parte 2

Continuación de la orquestación, exposición y validación del despliegue en Kubernetes.

[Ver video en YouTube](#)

Ruta sugerida de estudio: Contenedores 1 → Contenedores 2 → Kubernetes 1 → Kubernetes 2